Section 10: Stacks and Queues

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

C11: LeetCode DSA Foundations for Interviews

Section 10: Stacks and Queues

- **SLIDE 89: Section Intro, Stacks and Queues**
- **SLIDE 90: Stack and Queue Fundamentals with ArrayDeque**
- **SLIDE 91: Valid Parentheses**
- **SLIDE 92: Monotonic Stack: Next Greater and Next Smaller**
- **SLIDE 93: Worked Example: Daily Temperatures**
- **SLIDE 94: Queue for Level-Order Processing**
- **SLIDE 94B: Deque and Sliding Window Maximum**
- **SLIDE 95: Wrong vs Right: Stack and Queue Mistakes**
- **SLIDE 96: How to Present Monotonic Stack Solutions in an Interview**
- **SLIDE 97: Interview Practice: Stacks and Queues**
- **SLIDE 98: Section Cheat Sheet**

C11: LeetCode DSA Foundations for Interviews

Section 10: Stacks and Queues

SLIDE 89: Section Intro, Stacks and Queues

Section 10: Stacks and Queues

Stacks and queues are structurally simple: a stack is a last-in-first-out container, a queue is first-in-first-out. The data structures themselves are not the challenge. The skill this section builds is recognizing when a problem's structure maps onto LIFO or FIFO access patterns, because that recognition is the entire gap between a brute force and an optimal solution.

A stack is the right tool when the problem involves matching, nesting, ordering by recency, or finding the "next greater/smaller" element. A queue is the right tool when elements must be processed in arrival order or level by level.

What We'll Cover:

1. Stack and queue fundamentals using ArrayDeque in Java
2. Valid Parentheses as the introductory stack problem
3. Monotonic stacks for "next greater" and "next smaller" patterns
4. Worked example: Daily Temperatures with a decreasing monotonic stack
5. Queue for level-order processing and deque for sliding window maximum
6. How to present stack-based solutions in interviews

Where This Fits in the Course:

Two pointers (Section 6) and sliding window (Section 7) solve optimization problems on arrays by exploiting ordering or contiguity. Stacks solve structural problems: matching brackets, maintaining monotonic sequences, and finding relationships between elements based on their relative order. The deque application in this section connects back to sliding window (Section 7) by showing how to track the maximum element in a sliding window in O(1) per step. Queue-based BFS receives a brief introduction here and is covered in full in C6 (Trees/Graphs/DP).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 90: Stack and Queue Fundamentals with ArrayDeque

Stack: Last In, First Out (LIFO)

The most recently added element is the first one removed. Think of a stack of plates: you add to the top, you remove from the top.

Core operations, all O(1):

- `push(x)`: add element to the top
- `pop()`: remove and return the top element
- `peek()`: view the top element without removing it
- `isEmpty()`: check if the stack has no elements

Queue: First In, First Out (FIFO)

The earliest added element is the first one removed. Think of a line at a counter: first person in line is served first.

Core operations, all O(1):

- `offer(x)`: add element to the back
- `poll()`: remove and return the front element
- `peek()`: view the front element without removing it
- `isEmpty()`: check if the queue has no elements

Java Implementation: Use ArrayDeque for Both

```
// Stack usage
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
stack.push(30);
int top = stack.pop();    // 30
int next = stack.peek();  // 20

// Queue usage
Deque<Integer> queue = new ArrayDeque<>();
queue.offer(10);
```

```java
queue.offer(20);
queue.offer(30);
int front = queue.poll();  // 10
int next2 = queue.peek();  // 20
```

Why ArrayDeque, Not Stack or LinkedList

Java has a legacy `Stack` class. Do not use it. It extends `Vector`, which means every operation is synchronized (thread-safe but slower). In a single-threaded interview context, this overhead is unnecessary. `LinkedList` implements `Deque` but allocates a new node object per element, adding GC pressure.

`ArrayDeque` is backed by a resizable array, gives O(1) amortized performance for all operations, and is the recommended implementation for both stacks and queues in modern Java.

Implementation	Backing Structure	Thread Safe	Performance	Use In Interviews
`ArrayDeque`	Resizable array	No	O(1) amortized, cache-friendly	Yes, always
`Stack`	Synchronized `Vector`	Yes	O(1) but slower due to locking	No, legacy class
`LinkedList`	Doubly linked list	No	O(1) but extra allocation per node	No, unnecessary overhead

When to Use Which Data Structure:

Access Pattern	Data Structure	Recognition Signal
Process most recent first (LIFO)	Stack	Matching pairs, nesting, undo, “next greater/smaller”
Process in arrival order (FIFO)	Queue	BFS, level-order traversal, scheduling
Add/remove from both ends	Deque	Sliding window maximum, palindrome deque check

SLIDE 91: Valid Parentheses

The Introductory Stack Problem (LeetCode 20)

Given a string containing only the characters `(`, `)`, `{`, `}`, `[`, `]`, determine if the input string has valid bracket nesting.

Valid: every open bracket has a matching close bracket of the same type, and brackets are closed in the correct order.

Why a Counter is Not Enough

A simple counter (increment on open, decrement on close, check zero at end) works for a single bracket type like `()`. But it fails for multiple types. The string `([)]` has balanced counts for both bracket types, but the nesting is invalid: the `[` opens inside `(` but closes outside it.

A stack enforces ordering. When you encounter a close bracket, you check if the top of the stack is the matching open bracket. If it is, the nesting is correct for that pair. If it is not, the nesting is broken.

Full Java Solution:

```
public boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push(')');
        else if (c == '{') stack.push('}');
        else if (c == '[') stack.push(']');
        else {
            if (stack.isEmpty() || stack.pop() != c) {
                return false;
            }
        }
    }
```

```
        return stack.isEmpty();
    }
```

The technique: instead of pushing the open bracket and then checking the mapping on close, push the expected closing bracket directly. When you encounter a close bracket, pop and compare. If they match, the pair is valid. This eliminates the need for a separate mapping function.

Trace through `({[]})` (valid):

```
char '(' → push ')' → stack: [)]
char '{' → push '}' → stack: [), }]
char '[' → push ']' → stack: [), }, ]]
char ']' → pop ']', matches → stack: [), }]
char '}' → pop '}', matches → stack: [)]
char ')' → pop ')', matches → stack: []
Stack empty → return true
```

Trace through `([)]` (invalid):

```
char '(' → push ')' → stack: [)]
char '[' → push ']' → stack: [), ]]
char ')' → pop ']', does NOT match ')' → return false
```

The stack catches the invalid nesting immediately. The `]` on top of the stack tells us that the most recently opened bracket was `[`, but we encountered `)` before `]`. The ordering is wrong.

Three Failure Conditions:

Condition	Example	How Stack Catches It
Close bracket with no matching open	`)()`	`stack.isEmpty()` is true when close encountered
Wrong type of close bracket	`([)]`	`stack.pop() != c` detects type mismatch
Open brackets left unclosed	`(()`	`stack.isEmpty()` is false at end

Time O(n), space O(n) worst case (all open brackets).

SLIDE 92: Monotonic Stack: Next Greater and Next Smaller

The Pattern

A monotonic stack maintains its elements in strictly increasing or strictly decreasing order from bottom to top. When a new element violates the order, you pop elements until the order is restored. Each popped element has found its "answer" in the new element that caused the pop.

This pattern solves "next greater element" and "next smaller element" problems in O(n).

Decreasing Monotonic Stack (for Next Greater Element)

Elements on the stack decrease from bottom to top. When a new element is larger than the top, the top has found its next greater element: the new element itself. Pop and record the answer, then push the new element.

Why Store Indices, Not Values

Pushing indices instead of values lets you compute distances ("how many positions until a greater element?") and allows you to look up the actual value via `arr[index]` when needed. Always push indices on a monotonic stack unless the problem explicitly asks only for values.

Template (Next Greater Element):

```
public int[] nextGreaterElement(int[] arr) {
    int n = arr.length;
    int[] result = new int[n];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && arr[i] > arr[stack.peek
```

```
()]) {
            int idx = stack.pop();
            result[idx] = arr[i];
        }
        stack.push(i);
    }
    return result;
}
```

For each element at index `i`: while the stack is not empty and `arr[i]` is greater than the element at the index on top of the stack, pop and record `arr[i]` as the next greater element for that popped index. Then push `i`.

Why This is O(n), Not O(n^2)

The while loop inside the for loop looks like it could be O(n) per iteration, making the total O(n^2). But each element is pushed onto the stack exactly once and popped at most once. Across all iterations of the outer loop, the total number of push operations is n and the total number of pop operations is at most n. Total work: at most 2n = O(n).

This is the same amortized argument used for sliding window in Section 7: the inner loop is not O(n) per outer iteration. It is O(n) total across all outer iterations.

When to Recognize This Pattern:

- “Next greater element” or “next smaller element”
- “How many days until a warmer temperature”
- “Stock span” (how many consecutive days the price was less than or equal to today)
- “Largest rectangle in histogram”
- Any problem asking for the nearest element satisfying a comparison in one direction

Increasing vs Decreasing Stack:

Stack Order	What It Finds	Pop Condition	Use When
Decreasing (bottom to top)	Next greater element	New element > stack top	“Next warmer”, “next taller”, “stock span”
Increasing (bottom to top)	Next smaller element	New element < stack top	“Next cooler”, “largest rectangle in histogram”

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 93: Worked Example: Daily Temperatures

Problem: Daily Temperatures (LeetCode 739)

Given an array of daily temperatures, return an array where each element is the number of days you have to wait for a warmer temperature. If there is no future day with a warmer temperature, output 0.

Pattern Recognition: “How many days until a warmer temperature” is “next greater element with index distance.” This maps to a decreasing monotonic stack of indices.

Step-by-Step Approach:

Traverse the array left to right. Maintain a stack of indices where temperatures are in decreasing order. When the current temperature is greater than the temperature at the index on top of the stack, that top index has found its answer: the distance is `current index - popped index`.

Full Java Solution:

```
public int[] dailyTemperatures(int[] temperatures) {
    int n = temperatures.length;
    int[] result = new int[n];
    Deque<Integer> stack = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temperatures[i] > temperat
```

```
ures[stack.peek()]) {
            int prevDay = stack.pop();
            result[prevDay] = i - prevDay;
        }
        stack.push(i);
    }
    return result;
}
```

Trace through [73, 74, 75, 71, 69, 72, 76, 73]:

```
i=0: temp=73, stack empty → push 0
     stack: [0]                          (73)

i=1: temp=74 > 73(stack top) → pop 0, result[0] = 1-0 = 1
     stack empty, push 1
     stack: [1]                          (74)

i=2: temp=75 > 74(stack top) → pop 1, result[1] = 2-1 = 1
     stack empty, push 2
     stack: [2]                          (75)

i=3: temp=71 < 75 → push 3
     stack: [2, 3]                       (75, 71)

i=4: temp=69 < 71 → push 4
     stack: [2, 3, 4]                    (75, 71, 69)

i=5: temp=72 > 69(top) → pop 4, result[4] = 5-4 = 1
     temp=72 > 71(top) → pop 3, result[3] = 5-3 = 2
     temp=72 < 75(top) → stop popping, push 5
     stack: [2, 5]                       (75, 72)

i=6: temp=76 > 72(top) → pop 5, result[5] = 6-5 = 1
     temp=76 > 75(top) → pop 2, result[2] = 6-2 = 4
     stack empty, push 6
     stack: [6]                          (76)

i=7: temp=73 < 76 → push 7
     stack: [6, 7]                       (76, 73)

Indices 6 and 7 remain on stack → result stays 0 (no warmer day)
```

Result: [1, 1, 4, 2, 1, 1, 0, 0]

Key Observation at i=5:

When temperature 72 arrives, it triggers a cascade: it pops 69 (waited 1 day) and 71 (waited 2 days), but stops at 75 (not warmer). This cascade is what makes the monotonic stack powerful. Multiple elements find their answers in a single step, but the total pops across the entire traversal never exceed n.

Complexity:

- Time: O(n). Each of the n elements is pushed once and popped at most once. Total operations: at most 2n.
- Space: O(n). The stack stores at most n indices in the worst case (when temperatures are monotonically decreasing, like [76, 75, 74, 73]).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 94: Queue for Level-Order Processing

Queue and BFS

A queue processes elements in the order they arrive. This makes it the natural data structure for breadth-first search (BFS) and level-order traversal, where you process all elements at one level before moving to the next.

BFS Skeleton with Level Tracking:

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Deque<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        List<Integer> level = new ArrayList<>();
```

```
            for (int i = 0; i < levelSize; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.add(level);
        }
        return result;
    }
```

The `levelSize` variable captures the queue size at the start of each level. This lets you process exactly the nodes at the current level before moving to the next. Without it, you would mix nodes from different levels.

Full tree and graph BFS is covered in C6. This skeleton is included here so you recognize the queue pattern when you encounter it.

When to Use a Queue:

Signal	Why Queue	Example
"Level by level" or "layer by layer"	FIFO ensures level order	Binary tree level-order traversal
"Shortest path in unweighted graph"	BFS visits nearest nodes first	Shortest path in a grid
"Process in order of arrival"	FIFO preserves insertion order	Task scheduling simulation
"All nodes at distance K"	BFS expands one layer at a time	Find nodes K edges from a source

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 94B: Deque and Sliding Window Maximum

Deque: Double-Ended Queue

A deque (double-ended queue) supports O(1) insertion and removal at both the front and back. `ArrayDeque` in Java implements this. It combines the capabilities of a stack (push/pop at one end) and a queue (offer at back, poll from front).

Operation	Method	Time
Add to back	`offerLast(x)`	O(1)
Add to front	`offerFirst(x)`	O(1)
Remove from front	`pollFirst()`	O(1)
Remove from back	`pollLast()`	O(1)
View front	`peekFirst()`	O(1)
View back	`peekLast()`	O(1)

Sliding Window Maximum (LeetCode 239)

Given an array and a window size k, return the maximum value in each window position as the window slides from left to right.

Brute force: for each window, scan all k elements to find the max. O(n * k).

Optimal: maintain a deque of indices where the corresponding values are in decreasing order. The front of the deque always holds the index of the current window's maximum. O(n).

Two Maintenance Rules:

1. Remove from the front if the index has fallen outside the current window (expired)
2. Remove from the back if the value at the back index is less than or equal to the new element (dominated, can never be the maximum while the new element is in the window)

```
public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> deque = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
```

```
        while (!deque.isEmpty() && deque.peekFirst() < i - k
+ 1) {
            deque.pollFirst();
        }
        while (!deque.isEmpty() && nums[deque.peekLast()] <=
nums[i]) {
            deque.pollLast();
        }
        deque.offerLast(i);
        if (i >= k - 1) {
            result[i - k + 1] = nums[deque.peekFirst()];
        }
    }
    return result;
}
```

Trace through [1, 3, -1, -3, 5, 3, 6, 7] with k=3:

```
i=0: val=1,  remove dominated (none), add 0    deque:[0]          (1)
i=1: val=3,  remove dominated: 0(1<=3), add 1  deque:[1]          (3)
i=2: val=-1, remove dominated (none), add 2    deque:[1,2]        (3,-1)
     window [0..2], max = nums[1] = 3          result: [3]

i=3: val=-3, remove expired (none), add 3      deque:[1,2,3]      (3,-1,-3)
     window [1..3], max = nums[1] = 3          result: [3,3]

i=4: val=5,  remove expired: 1 < 4-3+1=2 → remove 1
     remove dominated: 3(-3<=5), 2(-1<=5)
     add 4                                     deque:[4]          (5)
     window [2..4], max = nums[4] = 5          result: [3,3,5]

i=5: val=3,  remove dominated (none), add 5    deque:[4,5]        (5,3)
     window [3..5], max = nums[4] = 5          result: [3,3,5,5]

i=6: val=6,  remove dominated: 5(3<=6), 4(5<=6)
     add 6                                     deque:[6]          (6)
     window [4..6], max = nums[6] = 6          result: [3,3,5,5,6]

i=7: val=7,  remove dominated: 6(6<=7), add 7  deque:[7]          (7)
     window [5..7], max = nums[7] = 7          result: [3,3,5,5,6,7]
```

Result: [3, 3, 5, 5, 6, 7]

Time O(n): each element is added to the deque once and removed at most once. Space O(k): the deque holds at most k indices.

This problem connects sliding window (Section 7) with deque. The window slides as before, but tracking the maximum requires the deque to efficiently discard dominated elements.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 95: Wrong vs Right: Stack and Queue Mistakes

Mistake 1: Using ArrayList as a Queue

```
// Wrong: ArrayList.remove(0) shifts all elements, O(n) per c
all
List<Integer> queue = new ArrayList<>();
queue.add(10);
queue.add(20);
int front = queue.remove(0);  // O(n) shift
```

This turns O(n) BFS into O(n^2) because each `remove(0)` shifts all remaining elements left.

Right: Use `ArrayDeque` with `offer()` and `poll()`, both O(1).

Mistake 2: Popping from an Empty Stack

```
// Crashes with NoSuchElementException
Deque<Integer> stack = new ArrayDeque<>();
stack.pop();  // stack is empty
```

Right: Always check `isEmpty()` before `pop()` or `peek()`.

```
if (!stack.isEmpty()) {
    int top = stack.pop();
}
```

In Valid Parentheses, this check catches the case where a close bracket appears with no matching open bracket (e.g., the string starts with `)`).

Mistake 3: Storing Values Instead of Indices on Monotonic Stack

Wrong:

```
// Pushing values: cannot compute distances
stack.push(temperatures[i]);
// Later: how many days between this element and the one that
popped it?
// You don't know, because you lost the index information.
```

Right:

```
// Pushing indices: distance = current index - popped index
stack.push(i);
// Later:
int prevDay = stack.pop();
result[prevDay] = i - prevDay;
```

When the problem asks “how many steps until X”, you need index distance. Storing only values makes this impossible.

Mistake 4: Using Java’s Legacy Stack Class

```
// Wrong: Stack extends Vector, all methods are synchronized
Stack<Integer> stack = new Stack<>();
```

Right:

```
// ArrayDeque: no synchronization overhead, faster
```

```
Deque<Integer> stack = new ArrayDeque<>();
```

This matters less for correctness and more for showing the interviewer you understand modern Java idioms. Using `Stack` signals outdated knowledge.

Mistake 5: Claiming O(n^2) for Monotonic Stack

Wrong: "The outer for loop is O(n) and the inner while loop pops up to n elements, so the total is O(n^2)."

Right: "Each element is pushed once and popped at most once across the entire traversal. The total number of push operations is n and the total number of pop operations is at most n. The inner while loop is not O(n) per outer iteration; it is O(n) total across all iterations. The overall complexity is O(n)."

This is the amortized argument. The same reasoning applies to the sliding window inner loop from Section 7. If you explained it there, reference it here: "This is the same amortized argument as the sliding window shrink loop."

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 96: How to Present Monotonic Stack Solutions in an Interview

Introducing the Concept

When the problem asks for "next greater element" or "how many days until a warmer temperature":

"For each element, I need to find the next element that is greater. A brute force would check every subsequent element for each position, giving O(n^2). Instead, I can use a monotonic stack. I maintain a stack of indices where the corresponding values are in decreasing order. When I encounter a value greater than the top of the stack, I know the top has found its next greater element. I pop and record the answer. Each element is pushed and popped at most once, so the total is O(n)."

Explaining the Amortized O(n) Argument

This is the part candidates most often skip or get wrong. The interviewer may specifically ask “why is this O(n) if there’s a while loop inside a for loop?” Be prepared:

“The key insight is that the while loop does not run n times per outer iteration. Each of the n elements is pushed onto the stack exactly once during the entire traversal. An element can only be popped once. So the total number of pops across all iterations of the outer loop is at most n. The total work is at most n pushes plus n pops, which is O(n).”

If the interviewer is satisfied, move on. If they want more: “You can think of it this way: each element enters the stack once and leaves once. The amortized cost per element is O(1) for the push and O(1) for the pop.”

When to Draw the Stack State

Monotonic stack problems are easier to verify when you draw the stack contents at each step. In a whiteboard interview, sketch 3-4 steps of the trace showing the stack growing and then cascading pops. This takes 30-60 seconds and demonstrates that your solution handles cascading pops correctly (like the i=5 step in Daily Temperatures where temperature 72 popped both 69 and 71).

Connecting to Other Patterns

If the interviewer asks for variants: “The same monotonic stack template handles next greater, next smaller, stock span, and largest rectangle in histogram. The only difference is the comparison direction: I use a decreasing stack for next greater (pop when new element is larger) and an increasing stack for next smaller (pop when new element is smaller).”

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 97: Interview Practice: Stacks and Queues

Practice Prompt 1

“Given a string containing just the characters `(`, `)`, `{`, `}`, `[` and `]`, determine if the input string is valid.” (LeetCode 20)

Expected Walkthrough:

Step 1: "I will use a stack. For each open bracket, I push the expected closing bracket. For each close bracket, I pop from the stack and check if they match."

Step 2: Implement the solution. Mention the three failure conditions: close with empty stack, type mismatch on pop, unclosed brackets remaining.

Step 3: Trace through one valid example like `({[]})` and one invalid example like `([)]`.

Step 4: "Time complexity O(n), space complexity O(n) in the worst case when all characters are open brackets."

Why a counter does not work: If the interviewer asks, explain: "A counter tracks balance but not ordering. The string `([)]` has balanced counts for both bracket types but invalid nesting. A stack enforces the correct nesting order by making me close the most recently opened bracket first."

Practice Prompt 2 (Follow-Up)

"What if you need to find the minimum number of brackets to remove to make the string valid?"

Approach: Use a stack to track indices of unmatched brackets. Push index of every open bracket. On close bracket, if stack top is matching open, pop (they cancel). Otherwise push the close bracket's index. After processing, remaining indices in the stack are unmatched. Answer: stack size.

```
public int minRemoveToMakeValid(String s) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (c == '(') {
            stack.push(i);
        } else if (c == ')') {
            if (!stack.isEmpty() && s.charAt(stack.peek()) ==
'(') {
                stack.pop();
            } else {
                stack.push(i);
```

```
                }
            }
        }
        return stack.size();
    }
```

Common Fresher Mistakes on Stack/Queue Problems:

- Using if-else chains to match brackets when the push-expected-close technique is cleaner and less error-prone
- Not handling the empty stack case, causing a runtime exception on inputs like `)(`
- Attempting to solve bracket problems with counting alone, which fails on interleaved types
- Using recursion for problems that have a clean iterative stack solution (unnecessary space on the call stack)

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 98: Section Cheat Sheet

Stack vs Queue vs Deque Decision Table

Structure	Access Pattern	Java Implementation	Use When
Stack (LIFO)	Last in, first out	`Deque<T> stack = new ArrayDeque<>()`	Matching/nesting, undo, monotonic ordering
Queue (FIFO)	First in, first out	`Deque<T> queue = new ArrayDeque<>()`	BFS, level-order, process in arrival order
Deque	Both ends	`Deque<T> deque = new ArrayDeque<>()`	Sliding window maximum, double-ended access

Monotonic Stack Templates

Decreasing stack (next greater element):

```
Deque<Integer> stack = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && arr[i] > arr[stack.peek()]) {
        int idx = stack.pop();
        result[idx] = arr[i];       // or: i - idx for distan
ce
    }
    stack.push(i);
}
```

Increasing stack (next smaller element):

```
Deque<Integer> stack = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && arr[i] < arr[stack.peek()]) {
        int idx = stack.pop();
        result[idx] = arr[i];       // or: i - idx for distan
ce
    }
    stack.push(i);
}
```

Valid Parentheses Template

```
Deque<Character> stack = new ArrayDeque<>();
for (char c : s.toCharArray()) {
    if (c == '(') stack.push(')');
    else if (c == '{') stack.push('}');
    else if (c == '[') stack.push(']');
    else if (stack.isEmpty() || stack.pop() != c) return fals
e;
}
return stack.isEmpty();
```

Pattern Recognition Signals

Signal in Problem	Pattern	Key Technique
“Valid brackets”, “balanced parentheses”	Stack matching	Push expected close, pop and compare
“Next greater element”, “next warmer day”	Decreasing monotonic stack	Pop when new > top, push indices
“Next smaller element”	Increasing monotonic stack	Pop when new < top, push indices
“Stock span”, “days since last higher price”	Decreasing monotonic stack	Count pops or index distance
“Largest rectangle in histogram”	Increasing monotonic stack	Pop when new < top, compute area
“Sliding window maximum”	Deque (decreasing value order)	Remove expired front, dominated back
“Level-order traversal”, “BFS”	Queue	offer children, poll level by level
“Min removals for valid brackets”	Stack of indices	Track unmatched positions

Key Complexity Facts

Problem	Time	Space	Why
Valid Parentheses	O(n)	O(n)	Each char pushed/popped at most once
Next Greater Element	O(n)	O(n)	Amortized: n pushes + n pops total
Daily Temperatures	O(n)	O(n)	Same amortized argument
Sliding Window Maximum	O(n)	O(k)	Each element enters/exits deque once
BFS Level Order	O(V + E)	O(V)	Visit each node and edge once

How This Connects to Other Sections

- Section 7 (Sliding Window): the deque-based sliding window maximum combines the sliding window technique with deque operations. The amortized O(n) argument for the monotonic stack is identical to the sliding window inner-loop argument.

- Section 8 (Binary Search): both monotonic stack and binary search find boundaries, but in different contexts. Binary search finds a boundary in sorted static data. Monotonic stack finds "next greater/smaller" relationships in sequential data.
- Section 11 (Linked Lists): stacks and linked lists share a LIFO insertion pattern. Some problems (e.g., reversing a linked list) can be solved with either a stack or pointer manipulation. The pointer approach is O(1) space while the stack approach is O(n).
- Section 12 (Recursion): recursion uses the call stack implicitly. Every recursive solution can be converted to an iterative one using an explicit stack. Understanding the stack data structure helps you reason about recursion depth and stack overflow.

Common Interview Mistakes

Mistake	Why It's Wrong	Better Approach
Using `ArrayList.remove(0)` as queue	O(n) per removal, turns BFS into O(n^2)	Use `ArrayDeque` with `poll()` for O(1)
Popping without `isEmpty()` check	Runtime exception on edge cases	Always guard pop/peek with isEmpty()
Storing values instead of indices	Cannot compute distances or positions	Push indices, look up values via `arr[idx]`
Using legacy `Stack` class	Synchronized overhead, outdated	`Deque<T> stack = new ArrayDeque<>()`
Claiming O(n^2) for monotonic stack	Misreads inner while loop as O(n) per iteration	Each element pushed once, popped once: O(n) total
Counter instead of stack for brackets	Fails on interleaved bracket types	Stack enforces ordering, not just balance

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina